DDA Line Drawing Algorithm - Detailed Note (Easy Version)

1. What Is DDA?

DDA means Digital Differential Analyzer.
It is a line drawing method used in computer graphics.

Goal:

- Start from point 1
- Move in small steps toward point 2
- Plot one pixel at each step

A monitor can only draw pixels, not a perfect mathematical line.
So DDA tells us exactly which pixels to light up.

---

2. Basic Idea in Simple Words

Given two points:

- Point A: (x0, y0)
- Point B: (x1, y1)

First compute the total movement:

- dx = x1 - x0
- dy = y1 - y0

Choose how many steps we need:

- steps = max(abs(dx), abs(dy))

Why max?

- If line is wide, x movement is larger
- If line is steep, y movement is larger
- We take the larger one so line has no gaps

Now compute per-step movement:

- x_inc = dx / steps
- y_inc = dy / steps

Then repeat:

- Plot pixel at round(x), round(y)
- x = x + x_inc

- y = y + y_inc

---

3. DDA Algorithm Steps

1. Read two endpoints: (x0, y0), (x1, y1)
2. Compute dx and dy
3. Compute steps = max(abs(dx), abs(dy))
4. If steps is 0, just plot one point and stop
5. Compute x_inc and y_inc
6. Set x = x0 and y = y0
7. Loop from 0 to steps:

- plot round(x), round(y)
- x = x + x_inc
- y = y + y_inc

---

4. Pseudocode

```
DDA(x0, y0, x1, y1):
    dx = x1 - x0
    dy = y1 - y0
    steps = max(abs(dx), abs(dy))

    if steps == 0:
        plot(x0, y0)
        return

    x_inc = dx / steps
    y_inc = dy / steps

    x = x0
    y = y0

    for i from 0 to steps:
        plot(round(x), round(y))
        x = x + x_inc
        y = y + y_inc
```

---

5. Example 1 (Diagonal Line)

Line from (2, 2) to (6, 6)

- dx = 4
- dy = 4
- steps = max(4, 4) = 4

- x_inc = 4 / 4 = 1
- y_inc = 4 / 4 = 1

Points generated:

- (2,2), (3,3), (4,4), (5,5), (6,6)

---

6. Example 2 (Vertical Line)

Line from (0, 2) to (0, 6)

- dx = 0
- dy = 4
- steps = max(0, 4) = 4
- x_inc = 0 / 4 = 0
- y_inc = 4 / 4 = 1

Points generated:

- (0,2), (0,3), (0,4), (0,5), (0,6)

So yes, this DDA form handles vertical lines.

---

7. Example 3 (Non-Integer Slope)

Line from (2, 2) to (6, 4)

- dx = 4
- dy = 2
- steps = 4
- x_inc = 1
- y_inc = 0.5

Float values while moving:

- (2.0, 2.0)
- (3.0, 2.5)
- (4.0, 3.0)
- (5.0, 3.5)
- (6.0, 4.0)

After rounding and plotting:

- (2,2), (3,2), (4,3), (5,4), (6,4)

---

8. File in This Project

Implementation file:

- tutorials/lineDrawing/DDA_line_draw.py

Main logic functions:

- dda_line(...): returns plotted line points
- dda_line_trace(...): returns per-step debug info

---

9. Controls in the App

Normal controls:

- Mouse click two points: draw line
- C: clear lines and states
- ESC: quit

Input controls:

- I: toggle typed input mode
- Type points in either format:
- x0 y0 x1 y1
- x0,y0 x1,y1
- Enter: draw from typed values

Debug controls:

- D: toggle debug mode
- SPACE: next debug step

Typed input also works in debug mode.

---

10. What Debug Mode Shows

Debug mode shows exactly what DDA is doing:

- dx, dy, steps
- x_inc, y_inc
- current step number
- current float x and y values
- rounded pixel that gets plotted

This helps students understand how floating values become integer pixels.

---

11. Time and Space Complexity

Let n = steps.

- Time complexity: O(n)
- Space complexity:

- O(1) if plotting directly
- O(n) if storing all points in a list

---

12. Advantages

1. Very easy to understand.
2. Easy to implement in code.
3. Good for teaching incremental drawing.
4. Works for all directions (with proper implementation).

13. Disadvantages

1. Uses floating-point arithmetic.
2. Rounding at each step may cause small visual error.
3. Bresenham is usually faster for integer pixel grids.

---

14. Important Test Cases

1. Same start and end point
2. Horizontal line
3. Vertical line
4. Steep line (abs(dy) > abs(dx))
5. Negative slope line

---

15. Small Exercises

1. Find DDA points for line (1,1) to (5,3).
2. For line (3,2) to (3,7), find dx, dy, steps, x_inc, y_inc.
3. Why do we use round(x), round(y) before plotting?
4. What happens if steps is set to abs(dx) only for a steep line?

16. Answers

1. (1,1), (2,2), (3,2), (4,2), (5,3)
2. dx = 0, dy = 5, steps = 5, x_inc = 0, y_inc = 1
3. Because screen pixels are integer positions, but DDA computes float positions.
4. Gaps can appear in steep lines because y changes faster than x.